

Analyse de l'Attrition

au sein d'une ESN

Classification automatique des causes de départ

Sommaire

01

Problématique

Comprendre pourquoi les employés quittent une ESN et construire un modèle prédictif

02

Analyse exploratoire

Exploration de 3 fichiers de données joints pour identifier les premières tendances d'attrition

03

Modélisation et optimisation

Entraînement et comparaison de 9 modèles, avec optimisation du seuil de décision pour maximiser le recall

04

Causes et recommandations

Interprétation globale et locale du modèle final pour guider les actions RH

Problématique

Contexte

Consultant Data Scientist pour TechNova Partners (ESN spécialisée en transformation digitale et SaaS).

Problème

Turnover anormalement élevé au sein de l'entreprise.

Mission

Identifier les causes racines des démissions.

Les données & la jointure

SIRH	Évaluations	Sondage
• Données RH de base • Âge, genre, salaire • Département, poste • Ancienneté	• Notes d'évaluation • Niveau hiérarchique • Heures supplémentaires • Augmentation salariale	• Satisfactions employé • Distance domicile-travail • Fréquence déplacements • Formations suivies

Jointure sur `sirh.id_employee` = `eval.eval_number` (sans "E_") = `sondage.code_sondage` →
1 470 employés | Jointure interne (inner join)

Insights clés — analyse exploratoire

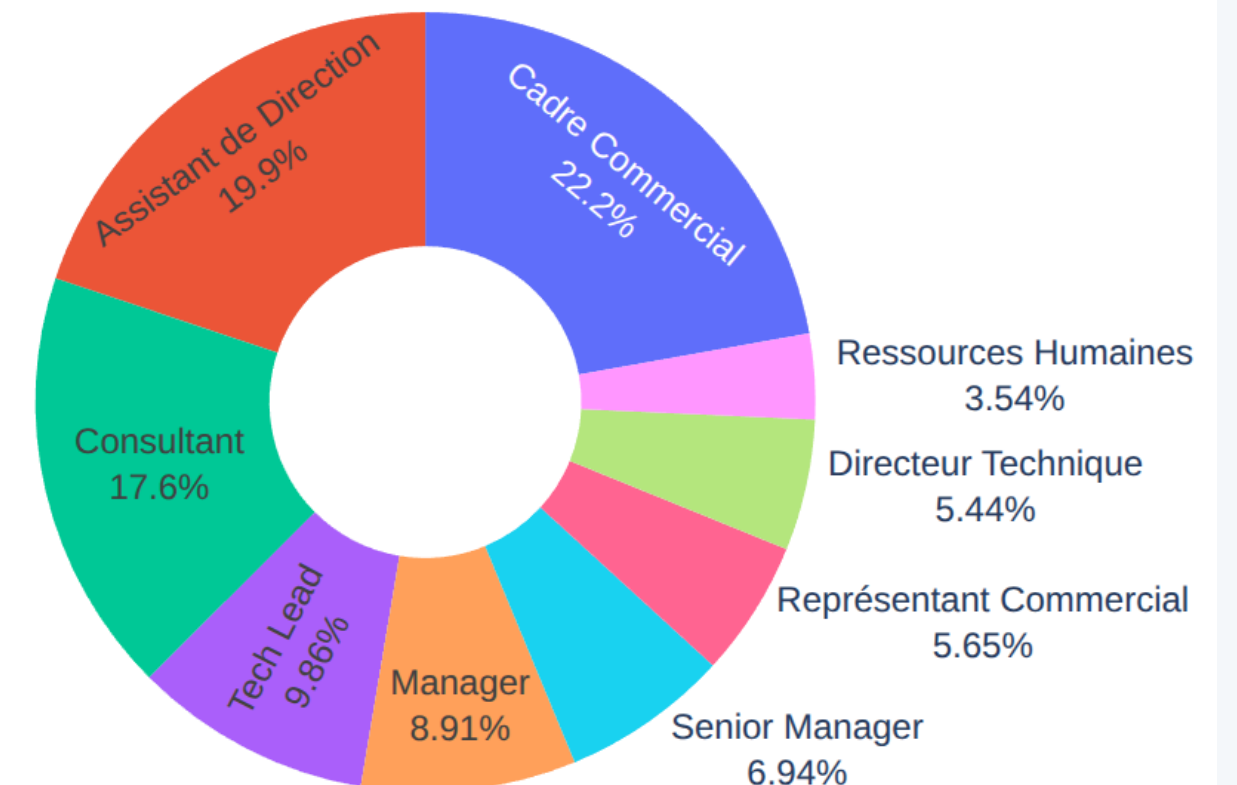
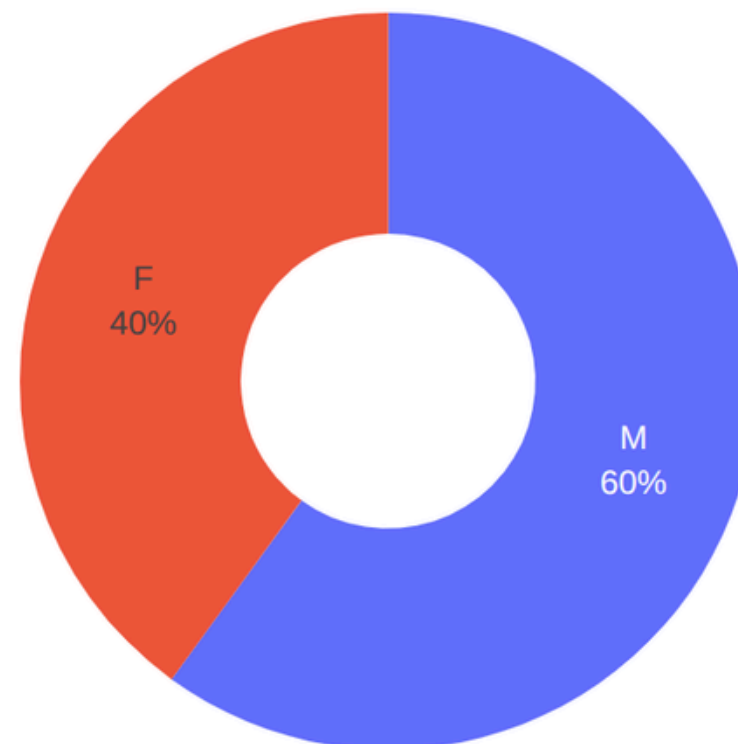
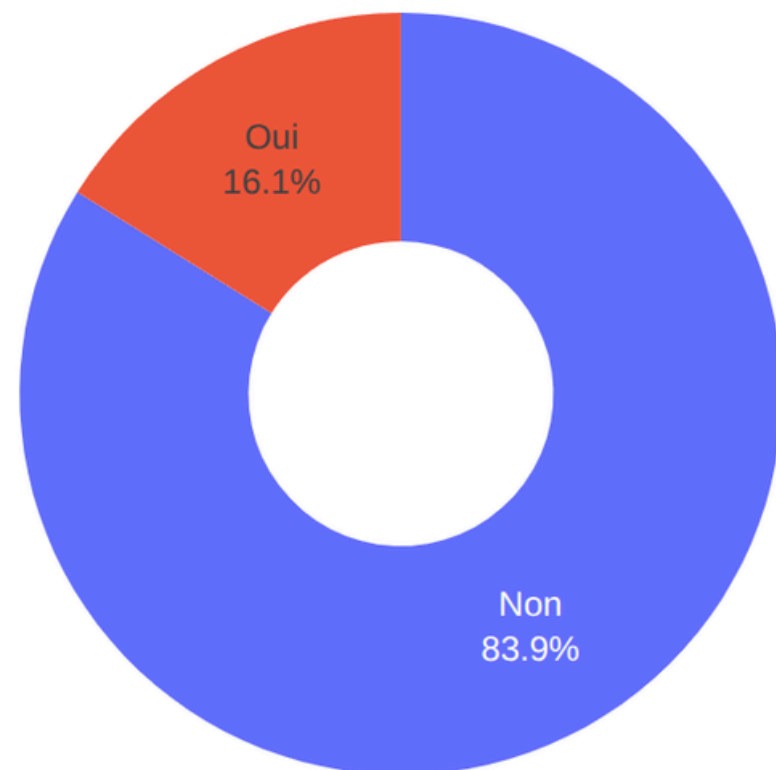
16%

taux d'attrition
(déséquilibre des classes)

1470 salariés, 34 colonnes

0 doublons, 0 valeurs manquantes, 0 valeurs abberantes

3 variables constantes : heures de travail, enfants, employee sous responsabilité



Feature engineering

3 nouvelles features, l'évolution de la performance sera l'un des facteurs important pour predire l'attrition.

```
df = df.copy()
#how many years worked outside
df['experience_externe'] = df['annee_experience_totale'] - df['annees_dans_l_entreprise']

#change in performance
df['evaluation_change'] = df['note_evaluation_actuelle'] - df['note_evaluation_precedente']

#speed of career progression
df['anciennete_avant_poste_actuel'] = df['annees_dans_l_entreprise'] -
df['annees_dans_le_poste_actuel']
```

Encoding

Encodage binaire, ordinal et One-Hot des variables catégoriques

```
# Binary
df["heure_supplementaires"] = df["heure_supplementaires"].map({"Oui": 1, "Non": 0})
df["genre"] = df["genre"].map({"F": 1, "M": 0})
df["a_quitte_l_entreprise"] = df["a_quitte_l_entreprise"].map({"Oui": 1, "Non": 0})

# Ordinal
df["frequence_deplacement"] = df["frequence_deplacement"].map({
    "Aucun": 0, "Occasionnel": 1, "Frequent": 2
})

# OneHot
nominal_cols = {"statut_marital": "is", "departement": "dep", "poste": "job", "domaine_etude":
"study"}
df = pd.get_dummies(df, columns=nominal_cols.keys(), prefix=nominal_cols.values(),
drop_first=False)
```

Corrélations

On a créé deux versions du dataset : `df`, où on a supprimé les colonnes avec de fortes corrélations, et `df_out`, où on a gardé toutes les colonnes

```
def drop_correlated_columns(df):  
    #we don't delete deps, jobs cause they are binary  
    cols_to_drop = ["niveau_hierarchique_poste", "note_evaluation_precedente",  
"note_evaluation_precedente"] #we could keep it for forest  
    return df.drop(columns=cols_to_drop, errors="ignore")  
  
df_out = df  
df = drop_correlated_columns(df)
```

X et Y split

Le jeu de données (1 470 lignes, 16 % de turnover) est trop petit pour une validation séparée : la validation croisée 10-plis sur l'entraînement suffit.

```
def create_X_y(df):  
    y = df["a_quitte_l_entreprise"]  
  
    X = df.drop(columns=["a_quitte_l_entreprise", "id_employe"],  
                 errors="ignore")  
  
    return X, y  
  
def split_X_y(df):  
    X, y = create_X_y(df)  
    X_train, X_test, y_train, y_test = train_test_split(  
        X, y, test_size=0.2, random_state=42, stratify=y  
    )  
    return X_train, X_test, y_train, y_test  
  
for name, df_vers in {'main': df, 'out': df_out}.items():  
    X_train, X_test, y_train, y_test = split_X_y(df_vers)  
    print(f"{name}: X_train={X_train.shape}, X_test={X_test.shape}")
```

Méthodologie de modélisation

1

Préparation

Encodage, feature engineering, nettoyage

2

Modèles testés

9 modèles : Dummy, LR, RF, XGBoost, SVC, Gradient Boosting...

3

Déséquilibre

Optimisation du seuil de décision (0.07 au lieu de 0.50)

4

Validation croisée

10 folds stratifiés — moyenne + écart-type par fold

Modèle Final

Gradient Boosting
sur df_out
seuil = 0.07

Recall test

89%

Recall CV moyen

85% ±9%

Précision

28%

Vrais départs captés

42 / 47

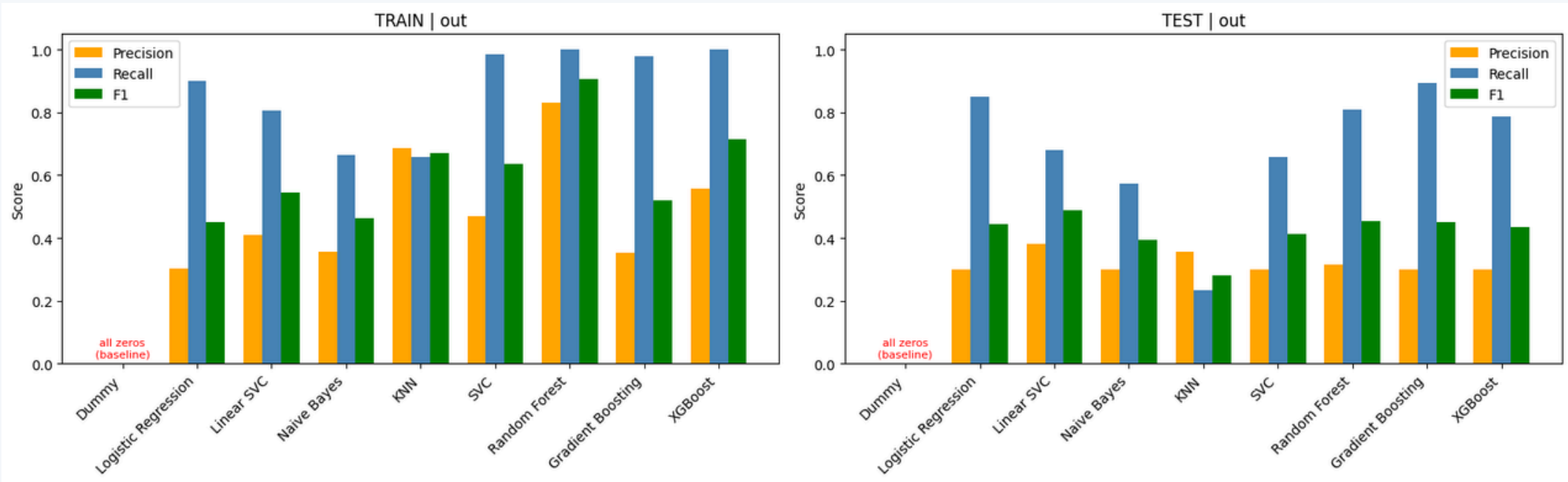
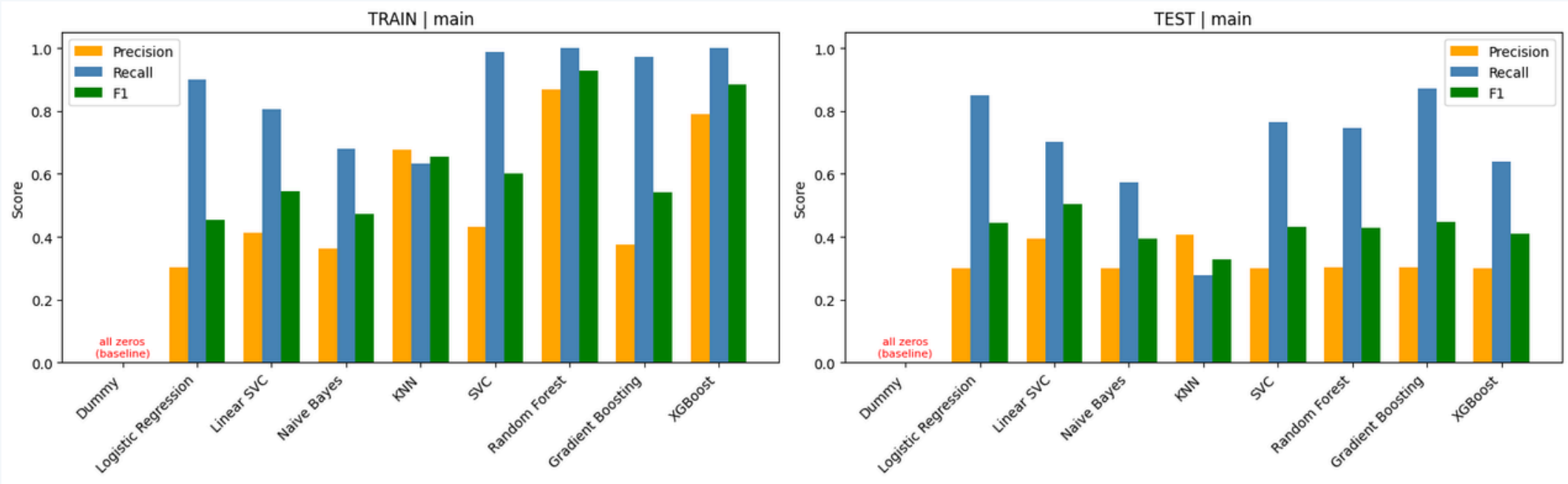
Métrique principale : Recall — manquer un départ coûte plus cher qu'une fausse alarme

Pourquoi recall ?

Métrique	Pourquoi pas optimal ?	Adapté à notre cas ?
Exactitude	Avec 84/16, prédire " <i>tous restent</i> " donne 84 % d'exactitude. Trompeur : ne reflète pas la détection des départs.	✗ Non
Précision	Minimise les faux positifs (coût faible), mais ignore les faux négatifs (coût élevé : manquer un départ).	✗ Non
F1	Bon compromis, mais suppose que faux positifs = faux négatifs en coût (faux ici).	⚠ Partiellement
AUC-ROC	Utile pour comparer les modèles, mais trompeur sur données déséquilibrées et n'optimise pas la détection des départs.	✗ Non
AUC-PR	Meilleur que AUC-ROC pour les données déséquilibrées, mais ne maximise pas directement le rappel .	⚠ Partiellement
Rappel	✅ Le bon choix : Un faux négatif coûte 10× plus qu'un faux positif. Objectif : capturer un maximum de départs . Le contexte métier justifie d'accepter des faux positifs.	✅ Oui

À reconsidérer F1 seulement si le RH a une capacité limitée à traiter les alertes.

Modèles

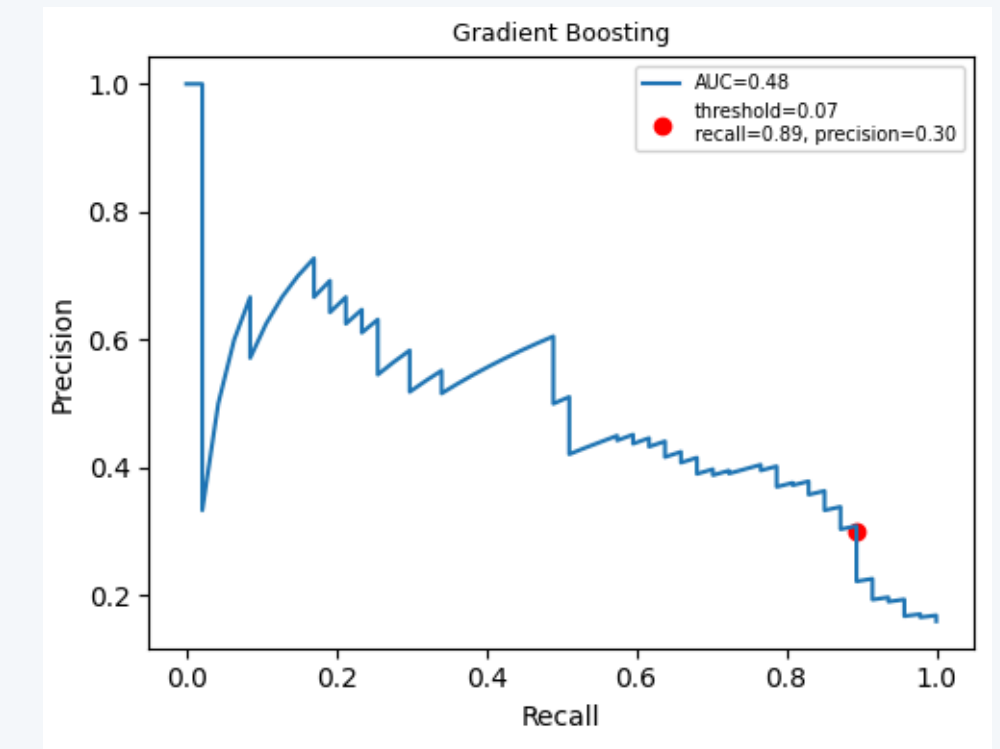
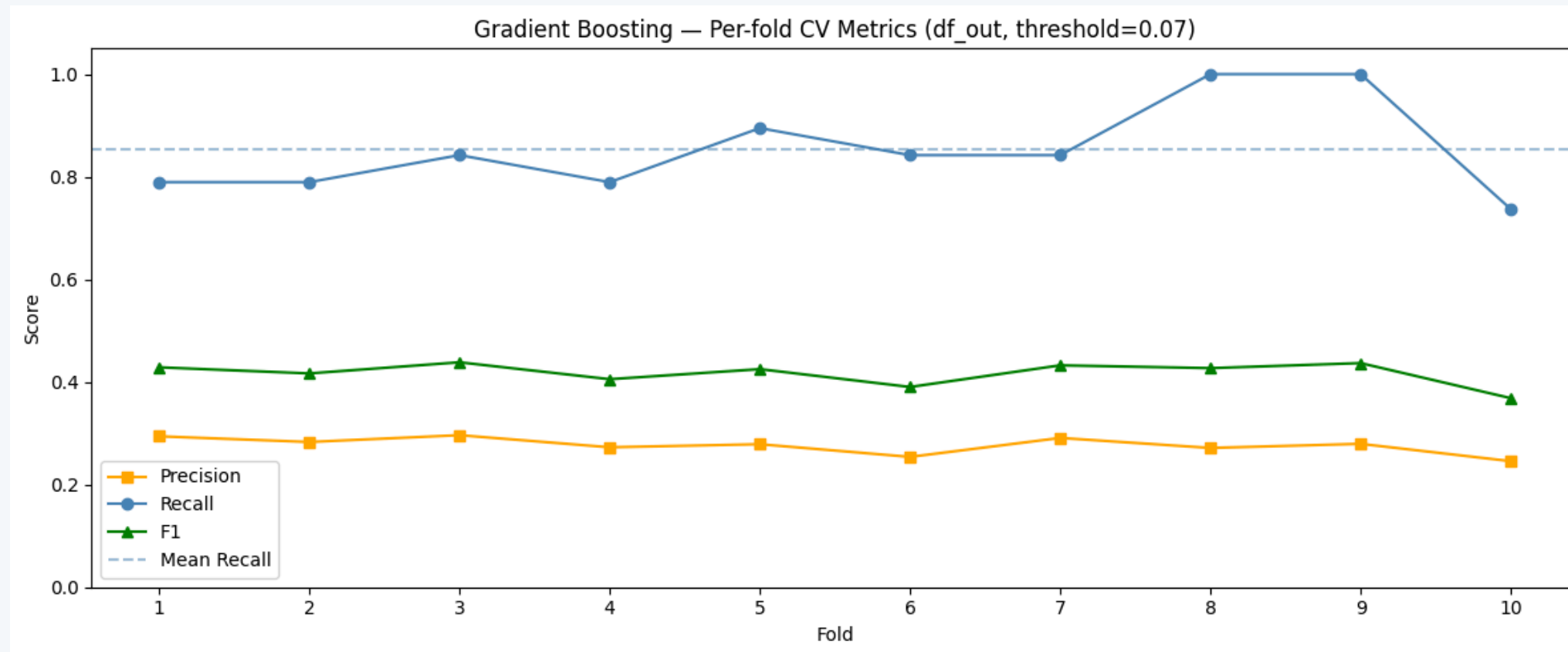


BEST MODELS SUMMARY		
MAIN		
Best Recall:	Gradient Boosting	(0.872)
Best F1:	Linear SVC	(0.504)
Best Precision:	KNN	(0.406)
OUT		
Best Recall:	Gradient Boosting	(0.894)
Best F1:	Linear SVC	(0.489)
Best Precision:	Linear SVC	(0.381)

Choix des modèles

Modèle	Type	Régularisation	Ensemble	Points clés
Dummy	Baseline	Non	Non	Prédit toujours la classe majoritaire.
Régression logistique	Linéaire	L1/L2 (optionnel)	Non	Modèle probabiliste, class_weight='balanced' pour le déséquilibre.
Linear SVC	Linéaire (SVM)	L2 (par défaut)	Non	Classifieur à marge maximale, sensible au <i>scaling</i> .
SVC	Non-linéaire (SVM)	L2	Non	Utilise des noyaux (ex : RBF) pour des frontières complexes.
Naive Bayes	Probabiliste	Non	Non	Suppose l'indépendance des features (hypothèse "naïve").
KNN	Basé sur la distance	Non	Non	Nécessite un <i>scaling</i> (ex : StandardScaler).
Random Forest	Bagging	Non	Oui	100+ arbres entraînés sur des sous-échantillons aléatoires.
Gradient Boosting	Boosting	Non	Oui	Arbres ajoutés séquentiellement pour corriger les erreurs.
XGBoost	Boosting optimisé	L1/L2	Oui	Version améliorée : régularisation intégrée, gestion des NA, parallélisation.

Gradient Boosting



```
Recall: 0.853 (+/- 0.089)
Precision: 0.277 (+/- 0.017)
F1: 0.417 (+/- 0.023)
```

--- Test ---

	precision	recall	f1-score	support
0	0.97	0.60	0.74	247
1	0.30	0.89	0.45	47
accuracy			0.65	294
macro avg	0.63	0.75	0.60	294
weighted avg	0.86	0.65	0.70	294

```
[[149 98]
 [ 5 42]]
```

Pourquoi Gradient Boosting ?

Régularisation (L1 et L2)

But : Éviter le surapprentissage en limitant la complexité du modèle.

L1 (Lasso):

Pénalise la valeur absolue des coefficients (peut mettre certains à 0 → sélection de features).

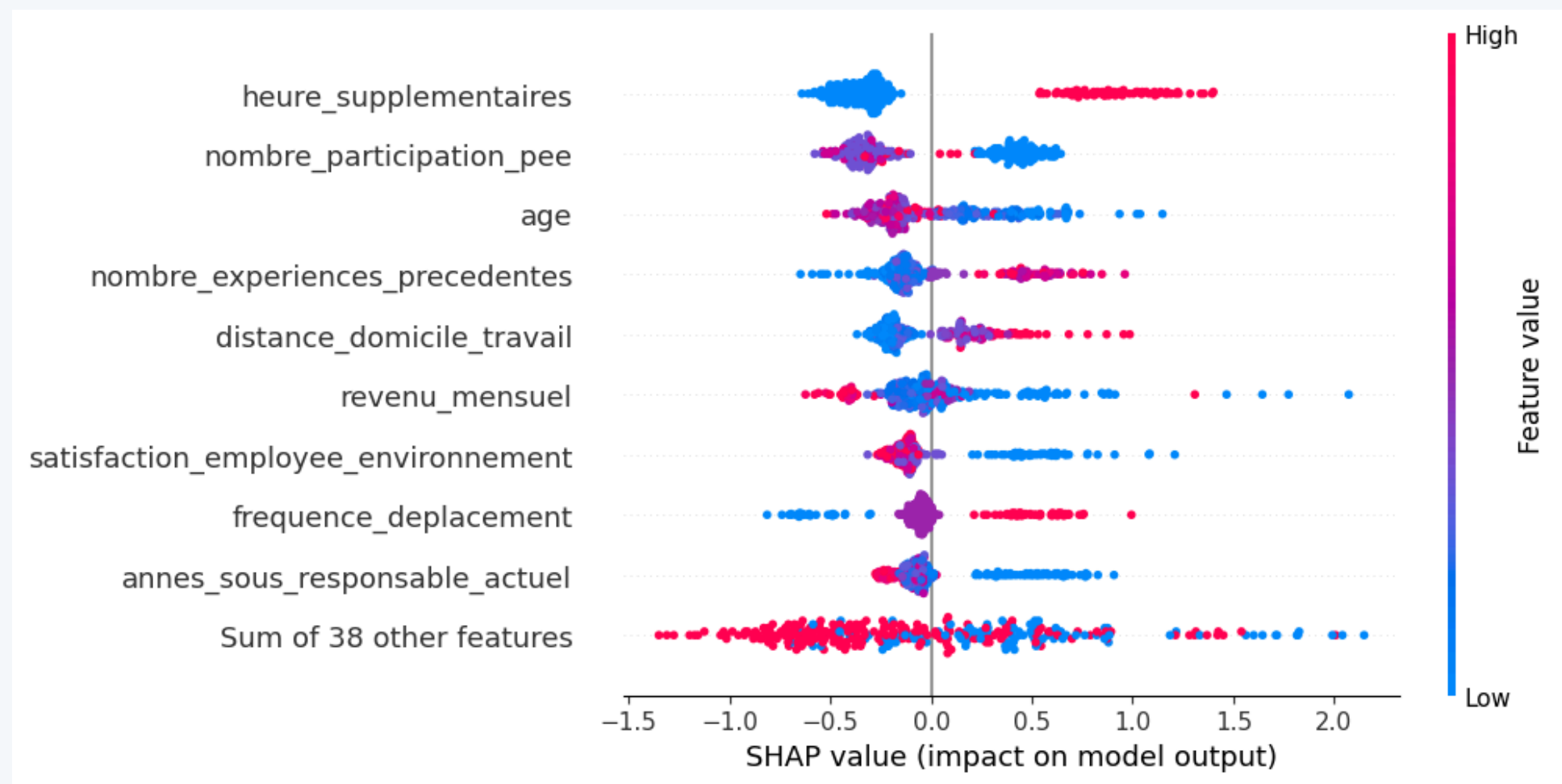
L2 (Ridge):

Pénalise le carré des coefficients (réduit leur amplitude sans les annuler).

Concept	Bagging (ex : Random Forest)	Boosting (ex : Gradient Boosting, XGBoost)
Principe	Entraîne des modèles en parallèle sur des sous-échantillons aléatoires (avec remplacement).	Entraîne des modèles séquentiellement , chaque nouveau modèle corrige les erreurs du précédent.
Combinaison	Moyenne les prédictions (réduction de la variance).	Additionne les prédictions (réduction du biais).
Surapprentissage	Très robuste (grâce à la diversité des sous-échantillons).	Risque de surapprentissage si trop d'itérations (contrôlé par learning_rate, max_depth).
Exemple	Random Forest : 100 arbres indépendants → vote majoritaire.	Gradient Boosting : Chaque arbre corrige les erreurs du précédent.

Gradient Boosting a gagné car il a exploité les motifs complexes du turnover (heures supplémentaires, satisfaction) sans être limité par une régularisation trop forte, tout en optimisant le rappel avec un seuil bas.

Causes d'attrition



	feature	importance	std
14	heure_supplementaires	0.087234	0.039748
15	augmentation_salaire_precedente	0.034043	0.010423
11	satisfaction_employee_equipe	0.027660	0.025265
41	study_Marketing	0.019149	0.006383
45	evaluation_change	0.012766	0.014113
16	nombre_participation_pee	0.010638	0.042818
17	nb_formations_suivies	0.010638	0.017154
0	age	0.010638	0.045881

Heures supplémentaires

Facteur #1 de loin. Les employés en surcharge de travail partent davantage.

Faibles augmentations

Un faible effort salarial signale un manque de reconnaissance.

Age

Les employés plus jeunes partent plus fréquemment, souvent en quête de nouvelles opportunités ou de progression.

Satisfaction d'équipe

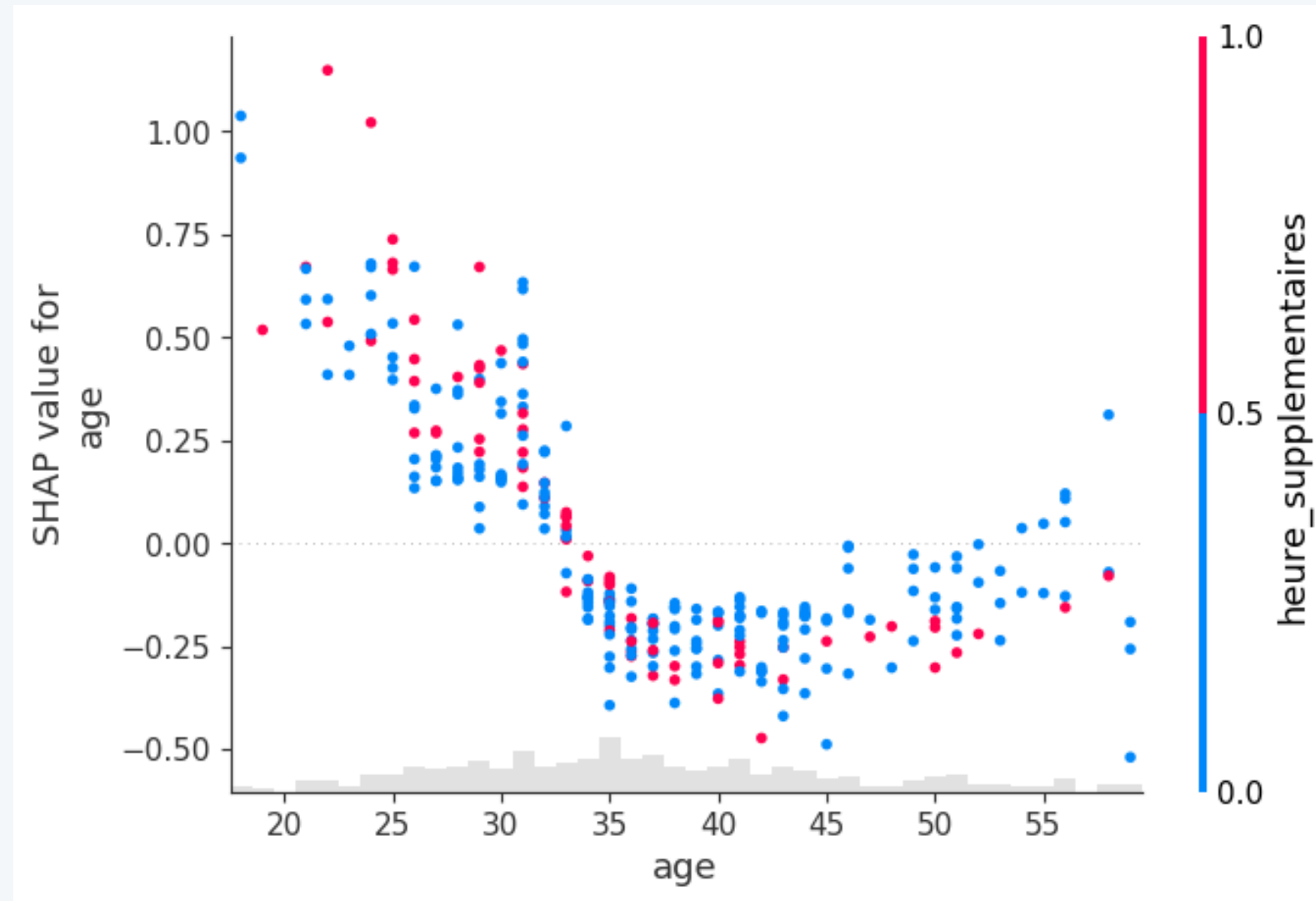
Un mauvais environnement humain pousse au départ.

Confirmé par SHAP Beeswarm et Permutation Importance — les deux méthodes convergent sur les mêmes facteurs.

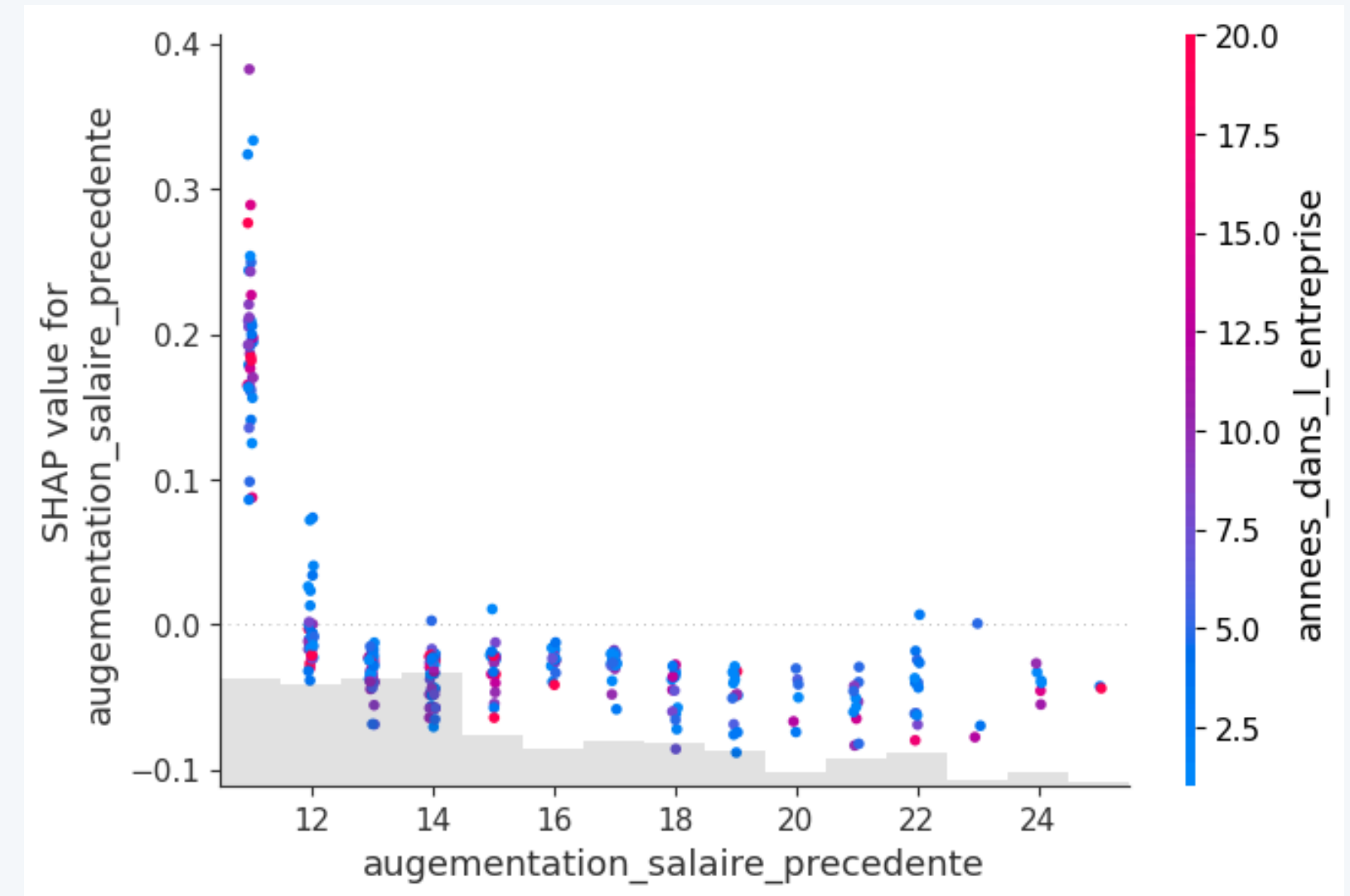
Permutation importance ou SHAP Beeswarm ?

Critère	Permutation Importance	SHAP Beeswarm
Principe	Mesure la baisse de performance (ex : précision, rappel) quand les valeurs d'une feature sont mélangées aléatoirement.	Mesure la contribution marginale de chaque feature à la prédiction, en comparant le modèle avec/sans cette feature (théorie des jeux coopératifs).
Approche	Modèle-dépendant : Utilise le modèle entraîné pour évaluer l'impact.	Modèle-agnostique : Fonctionne avec n'importe quel modèle (arbres, linéaire, etc.).
Ce qu'il mesure	Importance globale : Impact moyen de la feature sur la performance globale du modèle.	Impact directionnel : Montre si la feature augmente ou diminue la prédiction, et son ampleur.
Avantages	✓ Simple et intuitif.	✓ Capture les non-linéarités et interactions entre features.
	✓ Rapide à calculer.	✓ Montre la distribution des effets (ex : "satisfaction élevée → réduit le turnover").
Inconvénients	✗ Ignore les interactions entre features.	✗ Plus lent (surtout pour les modèles non-arbres).
	✗ Sensible aux corrélations entre features (une feature corrélée peut "voler" l'importance).	✗ Complexe à interpréter pour les débutants.
Cas d'usage	Comparer rapidement l'importance globale des features.	Comprendre comment et pourquoi une feature influence la prédiction (ex : "Pourquoi ce client part ?").
Visualisation	Barplot simple (ex : <code>sklearn.inspection.permutation_importance</code>).	Nuage de points (beeswarm) montrant la contribution de chaque feature pour chaque observation.
Exemple d'interprétation	<i>"Les heures supplémentaires sont la feature la plus importante pour le modèle."</i>	<i>"Une augmentation des heures supplémentaires augmente la probabilité de départ, surtout si la satisfaction est basse."</i>

Explications globales, SHAP Scatterplot

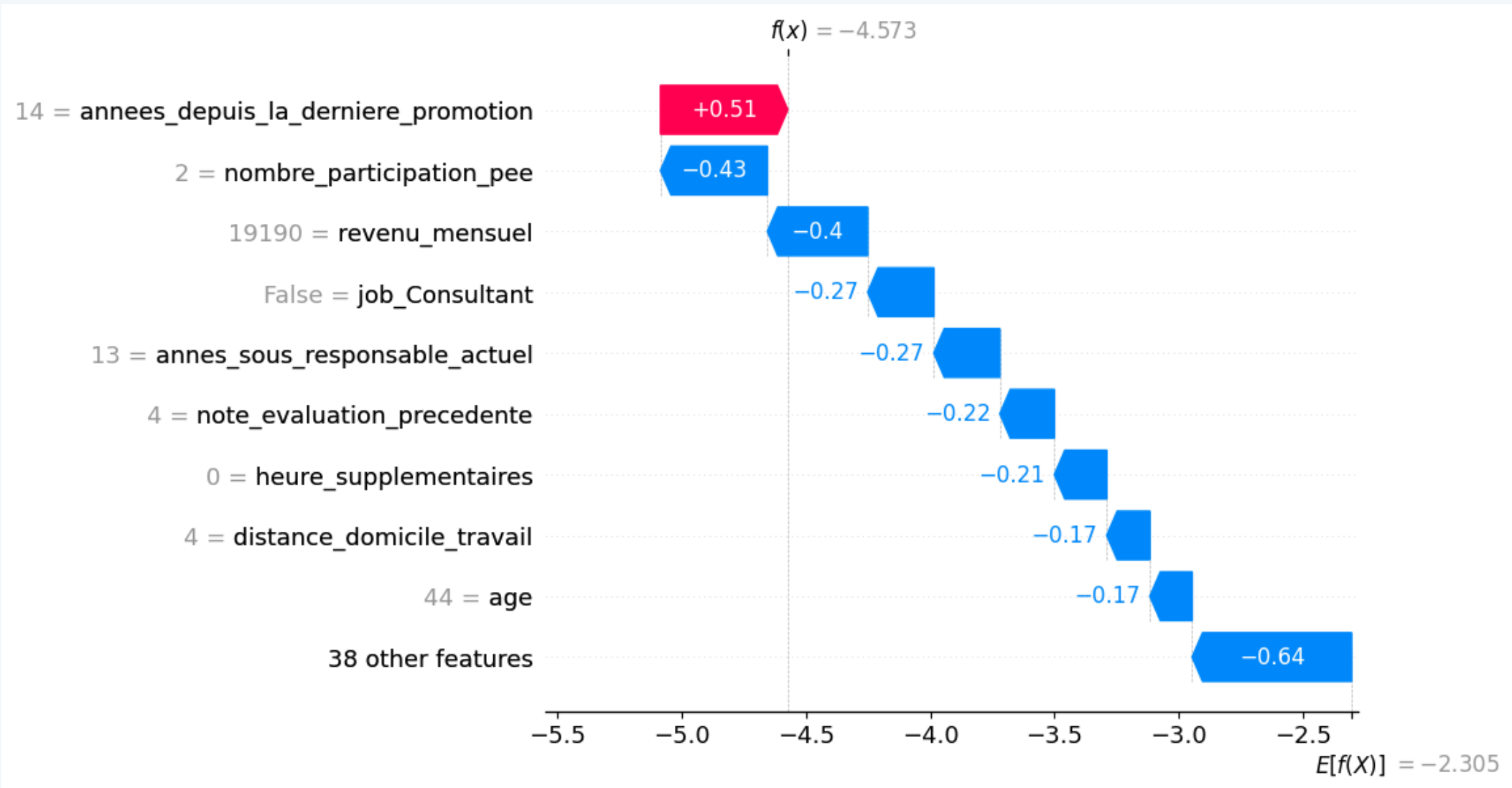


Les jeunes (<35 ans) partent davantage, indépendamment des heures sup. L'âge et l'overtime sont deux facteurs indépendants.



Une faible augmentation (~11-12%) pousse au départ, peu importe l'ancienneté. C'est le montant qui compte, pas le temps passé dans l'entreprise.

Explications individuelles, SHAP Waterfall



MAINTIEN PRÉDIT ✓

Employé B — est resté

↓ Pas d'heures supplémentaires

↓ Salaire élevé

↓ forte participation au PEE

Profil : stable, valorisé, bien intégré → rétention naturelle

Explications individuelles, SHAP Waterfall

DÉPART PRÉDIT ✓

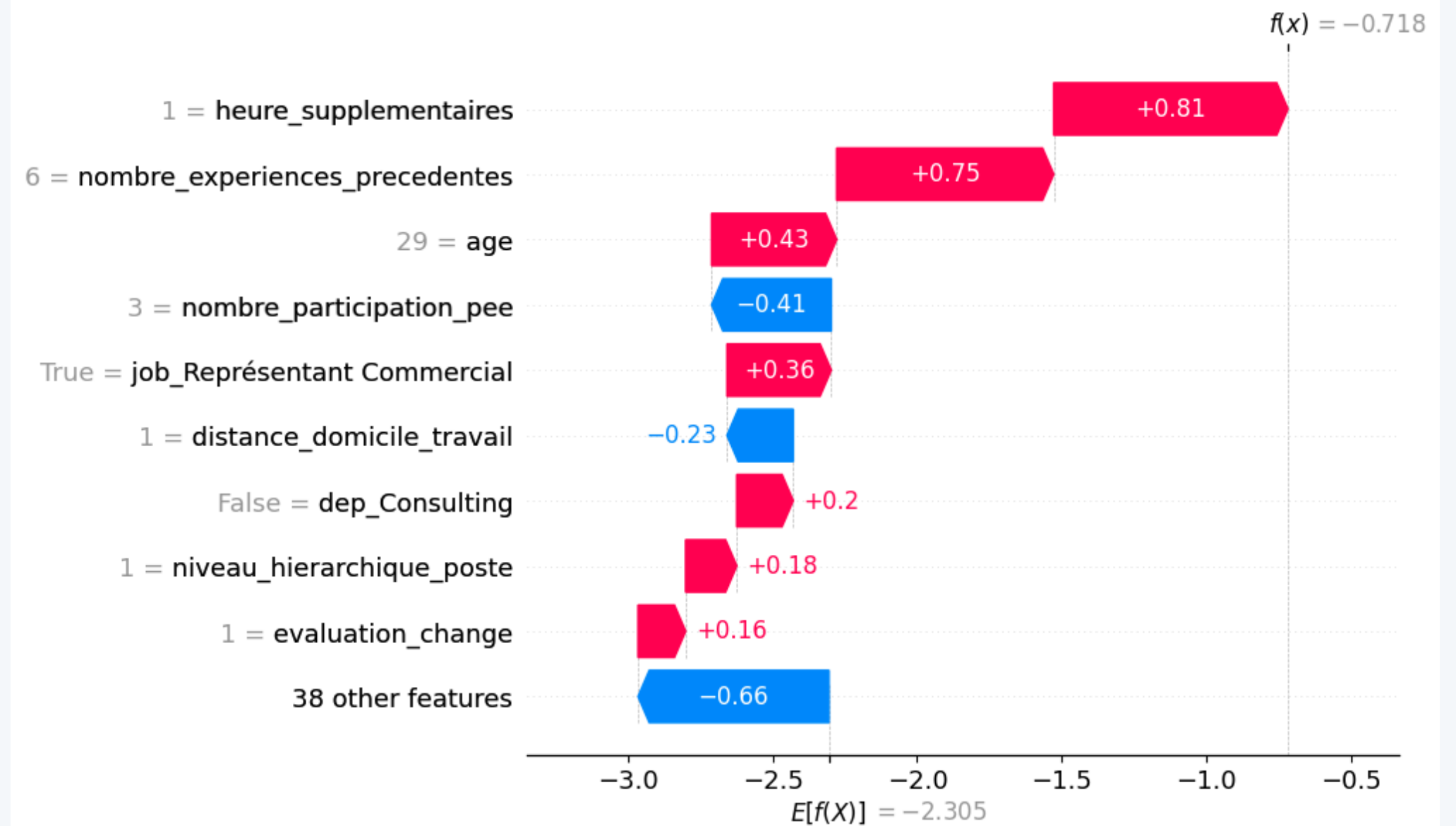
Employé A — a quitté l'entreprise

↑ Fait des heures supplémentaires

↑ Nombreuses expériences précédentes

↑ Jeune âge (mobilité élevée)

Profil : jeune, mobile, sous pression → départ prévisible



Recommandations RH

01

Auditer la politique des heures supplémentaires

Les limiter ou mieux les rémunérer — levier #1 identifié par le modèle

02

Revoir la politique d'augmentation salariale

Identifier et agir sur les employés avec de faibles augmentations répétées

03

Monitorer la satisfaction d'équipe

Surtout pour les jeunes employés et ceux avec une longue distance domicile-travail

04

Profil à risque élevé à surveiller

Jeune + heures supp. + long trajet + faible augmentation + plusieurs emplois précédents



Merci

Kseniia Sazonova
